Architecture Document

Emulator Fleet Sizing

Capacity Model

Fleet size is driven by peak concurrent missions, not user count or connected apps. Connected apps affect snapshot storage and session catalog size — they do not imply a running emulator per app.

The core formula:

```
required_slots = ceil(M × E × H)
```

Symbol	Meaning	Notes
M	Concurrent missions at peak	Primary fleet sizing driver
E	Avg. parallel emulators per mission	Typically 1–4 in practice
H	Headroom multiplier	1.2–1.4 for failures, rolling upgrades, and bursts

At 10,000 users, realistic peak concurrent missions depend heavily on usage patterns. Assume 2–5% active simultaneously (200–500 concurrent missions) as a starting point — not 10,000. Stampede scenarios are handled via admission control and queuing, not fleet sizing.

Per-Emulator Resources (Headless)

Resource	Per Slot
vCPU	2 (validate 1–2 for light workloads under load)
RAM	3–6 GiB guest + overhead
Ephemeral disk	5–20 GiB per slot (snapshot layers, app data — varies by app)

On a 32 vCPU / 128 GiB host, expect 4–8 emulator slots. Validate under load on the target instance family before committing.

Instance Requirements

Emulator workers must run on KVM-capable or bare-metal nodes. Standard burstable VMs (e.g. AWS t3, GCP e2) cannot reliably run nested Android emulators. Use dedicated node pools — for example, AWS c5.metal or m5.metal, or GCP n2 with nested virtualization enabled.

Hot vs. On-Demand Split

Two mechanisms serve different goals:

- Warm pool (per region, tier-stratified): Pre-booted idle slots for hot/warm sessions. Reduces provision time from ~30s (cold snapshot restore) to seconds. Size this to absorb steady-state P95 demand.

- On-demand autoscaling: Handles burst and overflow. Node bootstrap takes minutes — acceptable for queued/batch work, not interactive paths.

> **Recommendation:** Start with a warm pool covering ~120% of your steady-state P95 concurrent slot demand. Let autoscaling handle everything above that. Do not size the warm pool for stampede scenarios — use admission control and async UX instead.

Cost Model

Assume P = $1.50/host-hour:

```
monthly_compute ≈ N × P × 730
```

Scenario	Peak Slots	Slots/Host	Hosts (N)	Est. Monthly Compute
Steady-state (P95)	400	4	100	**~$110K**
Light load	40	4	10	~$11K

Add 15–25% for storage, egress, control plane, and observability. Use spot/preemptible instances for background health verification and batch work — reserve on-demand capacity for interactive mission paths.

> **Recommendation:** Express unit cost as $/concurrent emulator slot/hour from day one. This makes cost attribution and capacity tradeoffs legible as M and E change over time.

Snapshot Storage

S3 is the source of truth. Local SSD on each emulator host is ephemeral scratch space — disposable if the host dies. This keeps the backend stateless and horizontally scalable.

Current vs. Production

The prototype stores snapshots on local disk inside `.moboclaw_qcow2_sessions`. This breaks in production: if the host dies the snapshot is gone, and no two hosts can serve the same user's session. Production moves S3 to the source of truth with local SSD as a cache only.

Read Path — How Restore Actually Works

The flow on every mission start:

```
S3 (source of truth)
    ↓  pull session layer on demand (~5-15s for ~50-200 MB)
Local SSD on emulator host (ephemeral working copy)
    ↓  QEMU boots from local QCOW2 backing chain
```

```
Running emulator
    ↓  on checkpoint / teardown
Push delta back to S3
```

The local SSD is a cache, not storage. If the host dies mid-session, the last S3 checkpoint is intact. The scheduler detects the dead emulator, reschedules on a new host, and resumes from S3.

Three-Layer QCOW2 Backing Chain

A full session snapshot can be 2–8 GB. Pulling that from S3 on every restore blows the 30s latency target. QEMU's native QCOW2 backing chain solves this — each layer stores only the delta from its parent:

```
base.qcow2          (~4 GB)   — clean Android OS, rarely changes
  └── app.qcow2       (~500 MB) — app installed, changes on app update
        └── session.qcow2 (~50-200 MB) — user login state, changes often
```

On restore, only the session layer is pulled from S3 if the base and app layers are already cached on the host. That keeps the hot path well under 30 seconds.

Layer	Source of truth	Local cache	Eviction
base.qcow2	Baked into host AMI	Always present	Only on OS update
app.qcow2	S3	LRU on disk pressure	Popular apps pre-staged
session.qcow2	S3 (authoritative)	Pulled on demand, deleted after teardown	Immediately after teardown

> **Pre-stage:** Bake base.qcow2 into the host AMI. Pre-pull app layers for the top ~20 apps by usage frequency onto every emulator host. Only the session layer moves dynamically on every mission.

Write Path — Checkpoint & Teardown

```
1. Pause emulator
2. Commit session layer delta → new session.qcow2
3. Push to S3
   └ async (non-blocking) for mid-mission checkpoints
   └ sync (blocking) on final teardown — do not release slot until confirmed
4. Update metadata DB: new S3 key + etag + timestamp
5. Resume emulator or release slot
```

Compression & Cost

QCOW2 has built-in compression but zstd is meaningfully better. Convert before upload:

```
qemu-img convert -c -o compression_type=zstd session.qcow2 session-compressed.qcow2
```

At scale, rough storage cost estimate:

Sessions	Avg compressed size	Total	Est. S3 cost/mo
50K sessions	~100 MB	~5 TB	~$115/mo
150K sessions	~100 MB	~15 TB	~$345/mo

S3 Intelligent-Tiering auto-moves cold sessions (30+ days idle) to cheaper storage classes with no manual management needed. Deduplication across users of the same app (identical app layers) can further reduce storage materially.

Failure Modes

- **Host dies mid-mission:** Last S3 checkpoint is intact. Scheduler detects via health check, reschedules on a new host, pulls session layer from S3, resumes. User loses work only since last checkpoint.
- **S3 outage:** The primary risk. Mitigate with S3 Cross-Region Replication for hot sessions. During an outage, missions queue rather than fail permanently — session data is not lost.
- **Corrupt snapshot:** Validate etag and file integrity on pull. On corruption, fall back to the app layer and trigger a re-auth flow rather than hard-failing the mission.

Retrieval Latency Targets

Path	Expected latency	Mechanism
Hot session, app cached on host	< 10s	Pull session layer only (~50–200 MB)
Warm session, app not cached	10–30s	Pull app + session layers from S3
Cold session, full restore	30–60s	App pull + session pull + cold boot

Session Health at Scale

Steady-State Check Volume

```
100K sessions
├── Hot  (20%) → 20K checks/day  → ~14 checks/min
├── Warm (50%) → 50K checks/week →  ~5 checks/min
└── Cold (30%) → 0 background checks (on-demand only)
```

~19 concurrent checks/min at steady state. A dedicated pool of 5–10 health-check emulator slots handles this comfortably — kept separate from the mission pool so background verification never competes with live missions for capacity.

Scheduling

Checks are spread using per-session jitter so no two sessions pile up at the same time. Cold sessions are never checked in the background. A per-tick cap on the worker prevents a thundering herd if it restarts after downtime.

Health Check Behaviour per Tier

Not every check needs a full emulator boot. Always do the cheapest check first and only escalate when necessary.

Tier	Frequency	Method	Cost	On expiry
Hot	Daily	**Live verify —** boot from snapshot, open app, classify screen state, teardown. *Only tier with proactive emulator boots.*	~5–10s emulator time	Flag re-auth, update record
Warm	Weekly	**Metadata-only —** inspect TTL + last verified timestamp. *Live verify only if a mission is about to use the session.*	Free unless pre-mission	Return re_auth_required at mission time
Cold	Never (background)	**On-demand —** live verify only when a mission allocates the slot. *No proactive checks at all.*	Only when mission runs	Return re_auth_required

SLA per Tier

Tier	Freshness SLA	Expected expiry rate
Hot	Health known within 24h	~5–10%/day
Warm	Health known within 7 days	~30–40%/week
Cold	No guarantee — verify at mission time	~80%+

Failure Modes

The top 5 ways this system fails in production, with detection mechanism and recovery strategy for each.

1. Emulator Hang / ANR Mid-Mission

Failure: The emulator process is alive but unresponsive — adb commands time out, the app stops responding, or the Android watchdog triggers an ANR. The most common failure in practice.

Detection: Health monitor polls via consecutive failure threshold (`EMULATOR_MAX_HEALTH_FAILURES_BEFORE_REPLACE`). adb `shell getprop sys.boot_completed` hangs or returns nothing. ANR dialogs block the vision classifier.

Recovery: Health monitor auto-replaces the emulator. Mission task is re-queued from last checkpoint. If no checkpoint exists, the task restarts from the session snapshot. Emulator slot is released back to the pool after replacement.

2. Session Snapshot Corruption

Failure: The QCOW2 session file is present in S3 but corrupt — partial write on teardown, interrupted commit, or disk failure during snapshot capture. Mission fails to boot or boots into a broken state.

Detection: `qemu-img check` on pull before boot. Etag/checksum mismatch between S3 metadata and the downloaded file. Boot timeout — emulator never reaches `sys.boot_completed=1` within `EMULATOR_BOOT_COMPLETED_TIMEOUT_SECONDS`.

Recovery: Fall back to the app layer snapshot (one level up in the backing chain) and trigger re-auth flow. Update the session record to expired so the next mission gets a clean re_auth_required rather than a silent failure. Alert if corruption rate exceeds threshold — it may indicate a storage or commit bug.

3. Session Health Worker Falls Behind

Failure: The worker is running but processing slower than sessions are becoming due — backlog grows, next_check_at timestamps pile up, hot sessions go stale beyond their 24h SLA. Can happen after a deploy, a DB slowdown, or a spike in new sessions.

Detection: Monitor lag = `now - min(next_check_at)` for due sessions. Alert if this exceeds 2× the hot interval. Queue depth metric on due-but-unchecked sessions.

Recovery: `SESSION_MAX_HEALTH_CHECKS_PER_TICK` prevents the worker from flooding the emulator pool. Scale out health-check worker replicas — the per-tick cap is per-worker, so more workers = more throughput without overloading the pool. If lag is severe, temporarily deprioritise warm checks to catch up on hot sessions first.

4. Emulator Pool Exhaustion

Failure: All slots in the warm pool and on-demand capacity are occupied. New missions queue but emulator provisioning fails or stalls — either the node autoscaler hasn't caught up, or a bug is leaking slots (emulators not being torn down after mission completion or failure).

Detection: Pool utilisation metric — slots in use / total slots. Alert at >85%. Track slot lease age — any slot held for >2× the max mission duration is likely leaked. `DELETE /emulators/:id` teardown should always be called in a `finally` block to prevent leaks.

Recovery: For autoscaler lag: missions queue, async UX shows position and ETA. For slot leaks: a reaper job scans for leases older than the max duration and force-tears them down. For

sustained exhaustion: admission control returns retry-after rather than silently queueing indefinitely.

5. Identity Gate Timeout with No External Signal

Failure: A task hits an identity gate (OTP, biometric, payment approval) and waits for `POST /missions/:id/tasks/:taskId/approve`. The external signal never arrives — webhook delivery failed, the user ignored it, or the calling agent lost state. After `MISSION_IDENTITY_GATE_TIMEOUT_SECONDS` (300s), the task times out.

Detection: Task stays in `IDENTITY_GATE` state past the timeout threshold. Webhook delivery failures logged with response codes. A sweeper checks for tasks stuck in `IDENTITY_GATE` beyond the configured timeout.

Recovery: Task transitions to `FAILED` with reason `identity_gate_timeout`. Emulator slot is released. Mission is marked failed with the specific task and gate type in the response so the calling agent can retry with the correct context. The webhook payload should include enough information for the agent to resume without re-running completed tasks.

Observability

Metrics

Emulator Fleet

- `emulator_pool_utilisation` — slots in use / total. Most important operational metric.
- `emulator_provision_duration_seconds` — how long it takes to get a ready emulator; splitting by path shows whether you're staying on the fast pool path or falling back to slow boots.
- `emulator_health_failures_total` — counter by failure type. Spike = systemic issue.
- `emulator_slot_lease_age_seconds` — How long each in-use emulator slot has been checked out; extremely old ages often mean teardown/release didn't run (leaked capacity).

Session Health

- `session_health_check_lag_seconds` — `now - min(next_check_at)`. Canary for worker health.
- `session_expiry_rate` — per tier. Spike on hot = app session TTLs may have changed upstream.

Mission Pipeline

- `mission_duration_seconds` — P95/P99 are your SLO targets.
- `mission_task_state_transitions_total` — by from → to state. Shows where tasks get stuck.
- `identity_gate_timeout_total` — high rate = webhook delivery problem.
- `mission_queue_depth` — leading indicator of pool exhaustion.

Logs

Structured JSON on every state change. One correlation ID per mission, propagated to every log line it touches — emulator, session, task.

```
{"event": "emulator.health_failed", "emulator_id": "...", "failure_type":
"adb_timeout", "consecutive_failures": 2}
{"event": "session.check_completed", "user_id": "...", "app": "...", "tier": "hot",
"result": "logged_in", "duration_ms": 8100}
{"event": "task.state_changed", "mission_id": "...", "task_id": "...", "from":
"EXECUTING", "to": "IDENTITY_GATE", "gate_type": "OTP"}
{"event": "task.failed", "mission_id": "...", "task_id": "...", "reason":
"identity_gate_timeout"}
```

Alerts

Alert	Condition	Severity
Pool utilisation critical	emulator_pool_utilisation > 0.85 for 5min	**P1**
Emulator health failure spike	failure rate > 20/min	**P1**
Session worker lag	lag > 2 × hot interval	**P2**
Identity gate timeout spike	timeout rate > 10/min	**P2**
Cold provision latency	P95 cold provision > 60s	**P2**

On-Call Runbooks

P1: Pool Utilisation > 85%

Triage: (1) Is queue depth growing, or utilisation high but stable? (2) Check `emulator_slot_lease_age_seconds` for slots held far longer than max mission duration — these are leaks. (3) Is the autoscaler triggered and nodes coming up?

- **Legitimate load** → confirm autoscaler is active, enable `retry-after` admission control if not already on.
- **Slot leak** → reaper query: emulators where lease age > 2× max mission duration with no active mission. Force-teardown via `DELETE /emulators/:id`.
- **Autoscaler stuck** → manually scale node pool.

P2: Session Worker Lag > 2× Hot Interval

Triage: (1) Is the worker process running? (2) Is the due-sessions DB query slow — check index on next_check_at + tier. (3) Is the worker processing at all, just slowly?

- **Worker crashed** → restart. `SESSION_MAX_HEALTH_CHECKS_PER_TICK` prevents emulator pool flood on catch-up.

- **DB slow** → check for missing index on `next_check_at + tier` or table locks. Consider read replica for the due-sessions query.
- **Surge in due sessions** → temporarily process hot-tier first, skip warm until lag recovers.

P2: Identity Gate Timeout Spike > 10/min

Context: Each timeout holds an emulator slot for up to 5 minutes before release — a sustained spike drains the pool.

Triage: (1) Are webhooks being delivered — check response codes in logs. (2) Is the spike tied to a specific gate type (OTP, biometric, payment)? (3) Are slots releasing after timeout or is there a secondary leak?

- **Webhook delivery failing** → check endpoint health and retry config (`MISSION_WEBHOOK_CONNECT_TIMEOUT_SECONDS`). If endpoint is down, pause missions requiring that gate type.
- **Specific gate type** → check upstream auth provider. Route around if fallback exists.
- **Slots not releasing** → check that the `IDENTITY_GATE` sweeper is running and slot release is in a `finally` block.

Security

Emulator snapshots are full disk images. Anything the app persisted—session cookies, refresh tokens, OAuth artifacts, PII in caches, screenshots—is inside the blob. Treat every snapshot as sensitive at rest and whenever it moves (upload, restore, cross-region copy).

Encryption at Rest

- **S3 SSE-KMS** on the bucket with a customer-managed key (CMK) so you control rotation and revocation.
- **Per-user envelope encryption —** generate a data key per user, encrypt it with the CMK, store the encrypted data key in Postgres. A single compromised key exposes one user, not all of them.

Snapshots in transit (S3 → emulator host) are covered by TLS on the AWS SDK — no extra work needed.

Access Control

- **S3:** emulator worker IAM role gets `s3:GetObject` scoped to their region prefix only. No listing, no cross-region.
- **Metadata DB:** the `snapshots` table with `user_id → snapshot_id → S3 key` mapping is the real attack surface. Audit log on reads/writes. If someone can write to it, they can redirect a mission to another user's snapshot.
- **Emulator hosts:** `DELETE /emulators/:id` must explicitly delete the local snapshot copy — not just release the slot.

Snapshot Lifecycle

- Active snapshots replace in place on each checkpoint — no versioning needed.

- S3 lifecycle rule driven by `last_access_at` from the session tier system: move to cold storage at 90 days, delete at 180 days.

When a User Disconnects an App

This is the most security-sensitive operation — a valid session token may still be alive in the snapshot.

1. **Hard-delete the snapshot from S3**.
2. **Delete the session row and health history** from Postgres.
3. **If an emulator is running** with that session: terminate immediately, wipe local snapshot, release slot. Do not wait for the mission to finish.
4. **Fail any in-flight mission** with `session_revoked`.
5. **Write an audit log entry:** `user_id`, `app_package`, `snapshot_id`, `deleted_at`, `reason: user_disconnected`. Keep this even after the data is gone.